# Digital Assignment Lab - 1

**Course Name :-** Agile Development Process and Devops Lab

**Course code :-** ISWE406P

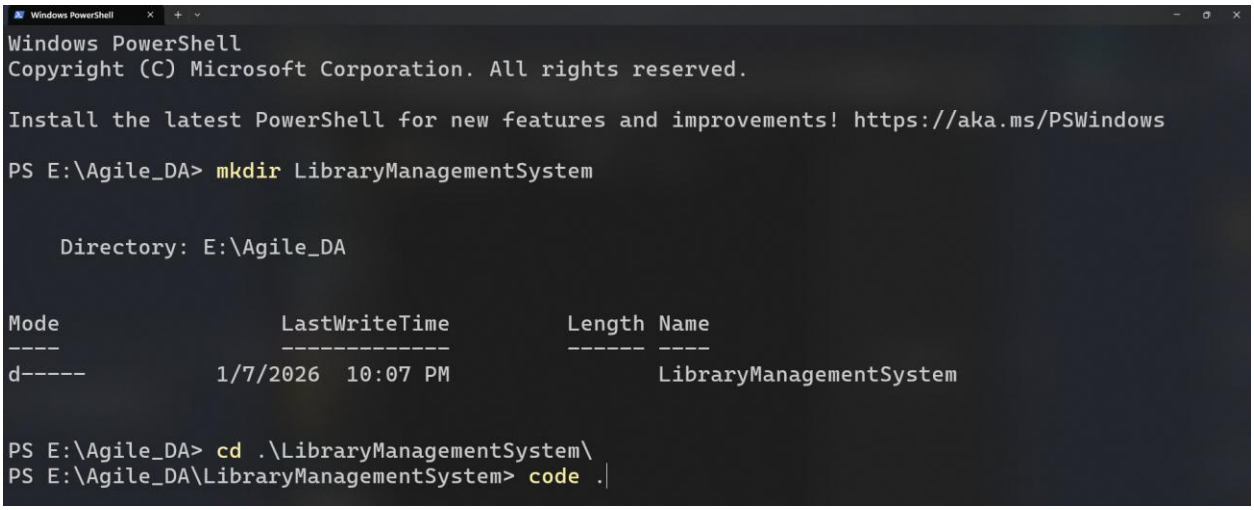
**Name :-** Kommi Druthendra

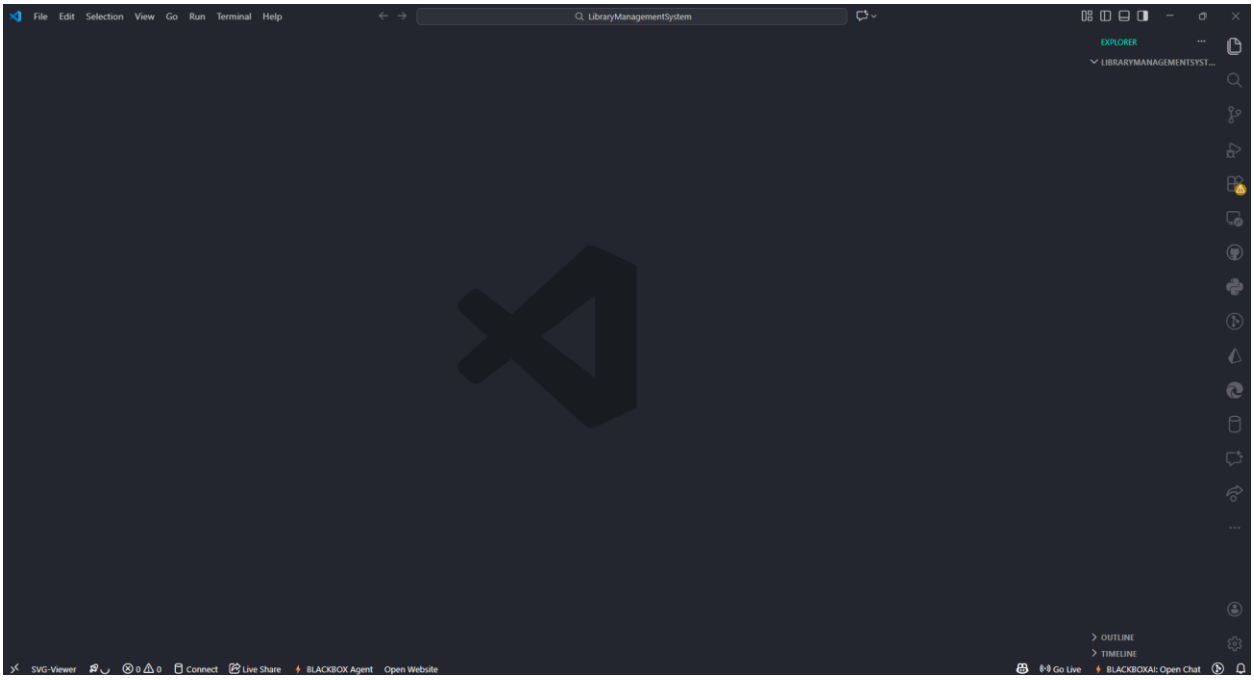
**Reg-No:-** 23MIS0213

**Case Study 1:** New Project Version Control Setup

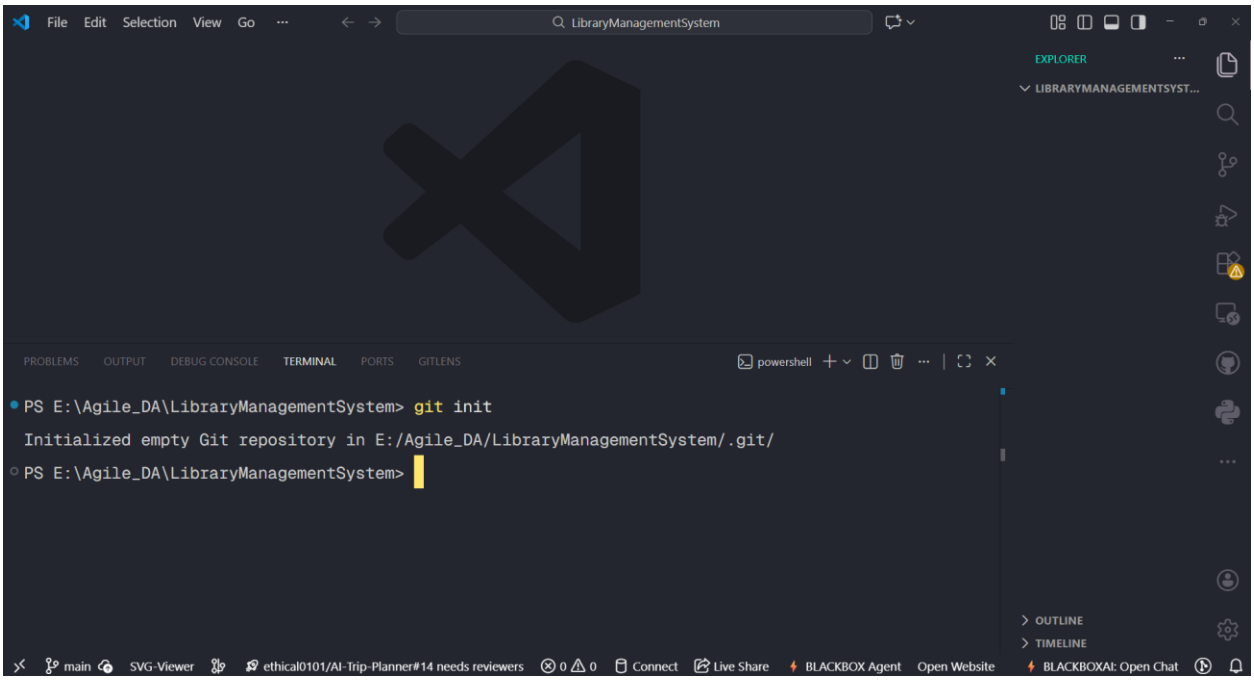
Scenario: You are assigned to develop a Library Management System. Before coding, your team decides to enable version control.

Create a project folder and open it in VS Code

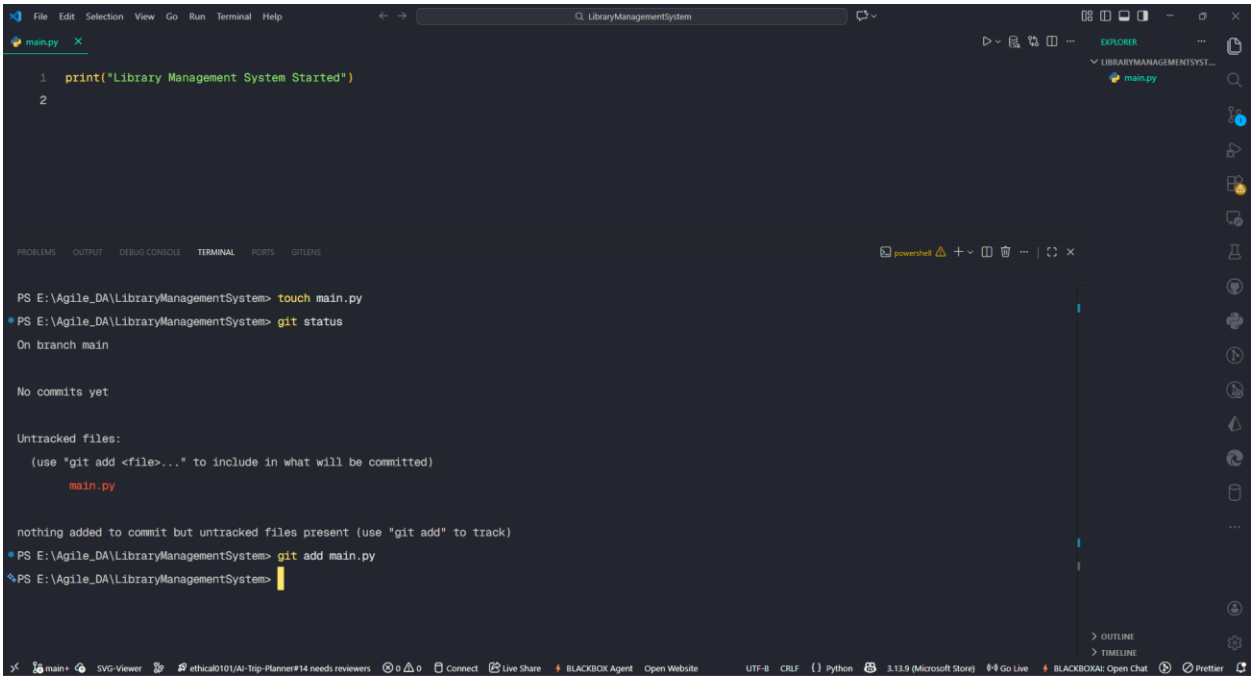


## Initialize a Git repository




## Create a Python file and track it using Git

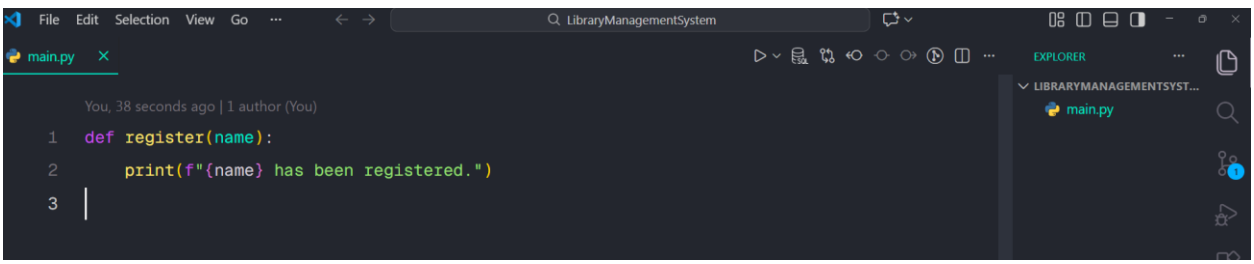


Commit the Initial version

```
PS E:\Agile_DA\LibraryManagementSystem> git commit -m "Initial commit"
[main (root-commit) 9cc5022] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 main.py
PS E:\Agile_DA\LibraryManagementSystem>
```

**Case Study 2:** Tracking Daily Code Changes Scenario: You are working on a Student Registration Module. You make small updates daily and need to record them.

Modify an existing file




Check file differences

```
PS E:\Agile_DA\LibraryManagementSystem> git diff
diff --git a/main.py b/main.py
index c7ca0e7..3c83581 100644
--- a/main.py
+++ b/main.py
@@ -1 +1,2 @@
-print("Library Management System Started")
+def register(name):
+    print(f"{name} has been registered.")
PS E:\Agile_DA\LibraryManagementSystem>
```

Stage and commit changes

```
PS E:\Agile_DA\LibraryManagementSystem> git add main.py
PS E:\Agile_DA\LibraryManagementSystem> git commit -m "Added student registration function"
[main 755d3f5] Added student registration function
 1 file changed, 2 insertions(+), 1 deletion(-)
PS E:\Agile_DA\LibraryManagementSystem>
```

View commit history

```
PS E:\Agile_DA\LibraryManagementSystem> git log --oneline
755d3f5 (HEAD -> main) Added student registration function
9cc5022 Initial commit
PS E:\Agile_DA\LibraryManagementSystem> git log
commit 755d3f5c08a107393af8c041c39c46b14a362b5e (HEAD -> main)
Author: ethical0101 <kommidruthendra2005@gmail.com>
Date:   Wed Jan 7 22:42:37 2026 +0530

    Added student registration function

commit 9cc502292818c20cef90d330702dee34b3b7e7f7
Author: ethical0101 <kommidruthendra2005@gmail.com>
Date:   Wed Jan 7 22:27:59 2026 +0530

    Initial commit
PS E:\Agile_DA\LibraryManagementSystem>
```

**Case Study 3:** Feature Development Using Branches Scenario: A new Login feature must be developed without disturbing main code.

Create a new branch

```
PS E:\Agile_DA\LibraryManagementSystem> git branch login-feature
PS E:\Agile_DA\LibraryManagementSystem>
```

## Switch Branch

```
PS E:\Agile_DA\LibraryManagementSystem> git branch login-feature
PS E:\Agile_DA\LibraryManagementSystem> git checkout login-feature
Switched to branch 'login-feature'
PS E:\Agile_DA\LibraryManagementSystem>
```

## Add login-related code

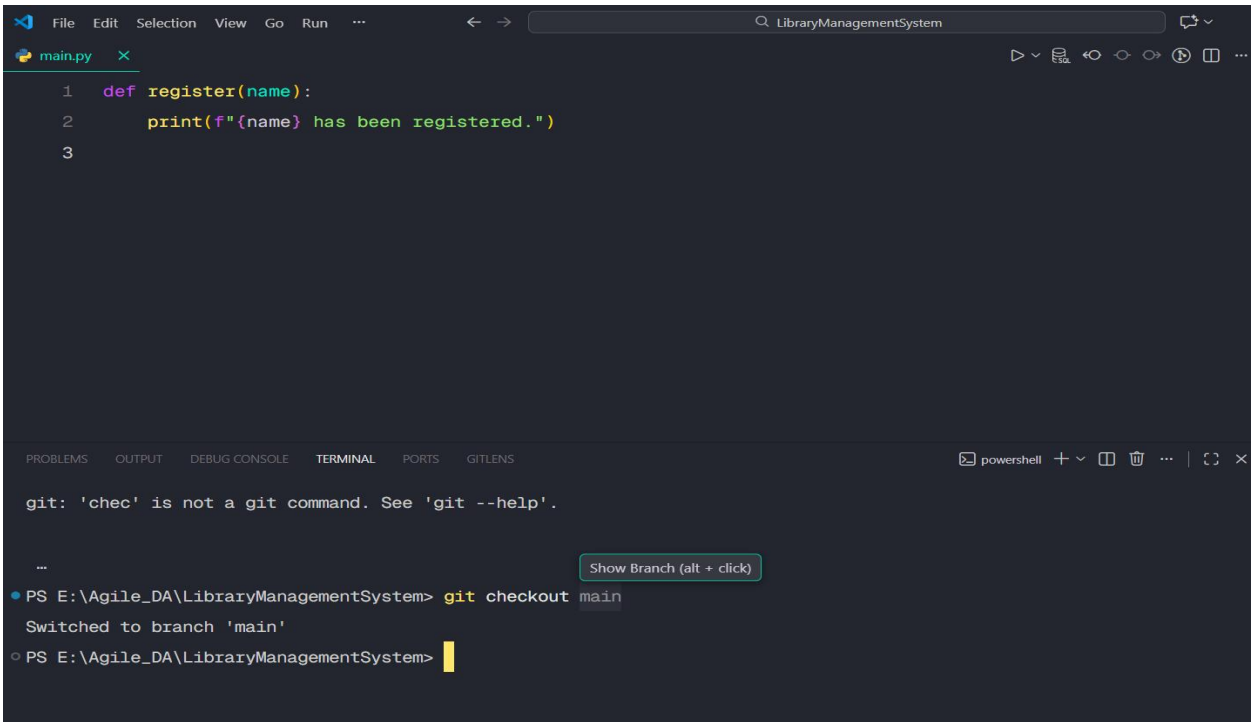
```python
def register(name):
    print(f"{name} has been registered.")

#Login Feature code
def login(user, password):
    # simple hardcoded credentials
    correct_user = "admin"
    correct_password = "1234"

    if user == correct_user and password == correct_password:
        print("Login successful")
    else:
        print("Login failed")

register("admin")
login("admin", "1234")
```

## Commit changes

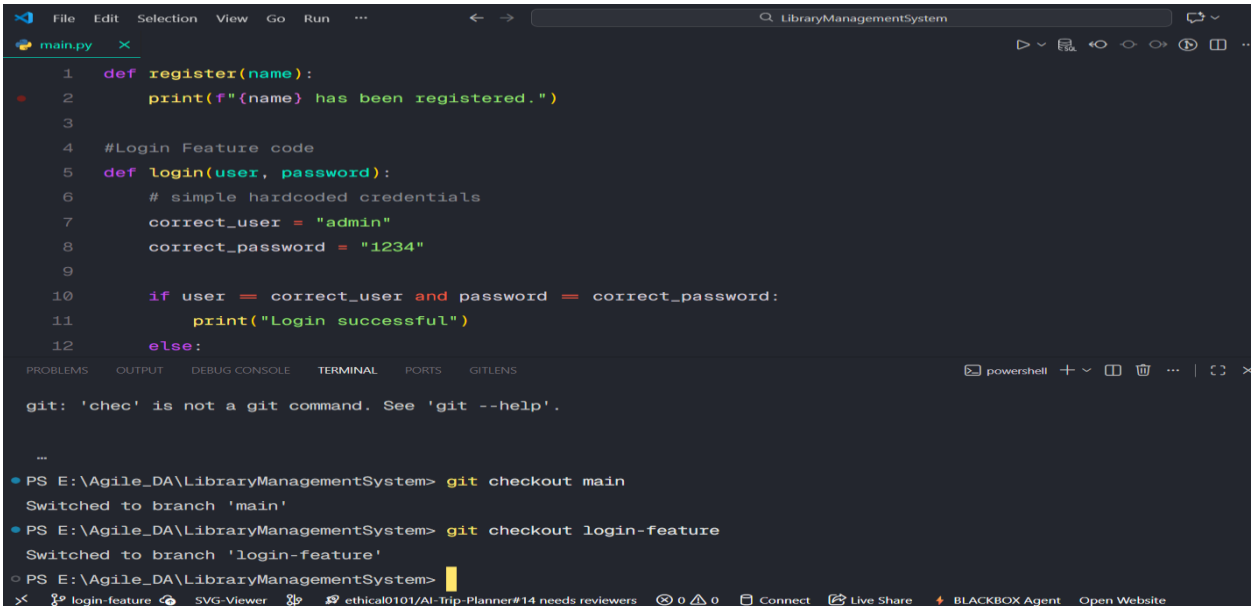
```
PS E:\Agile_DA\LibraryManagementSystem> git add main.py
PS E:\Agile_DA\LibraryManagementSystem> git commit -m "Added Login Feature Code"
[login-feature a783f4f] Added Login Feature Code
 1 file changed, 14 insertions(+)
PS E:\Agile_DA\LibraryManagementSystem>
```

# Switch between branches

## Main Branch




## login-feature Branch



**Case Study 4:** Merging Feature into Main Code Scenario: The login feature is tested and approved.

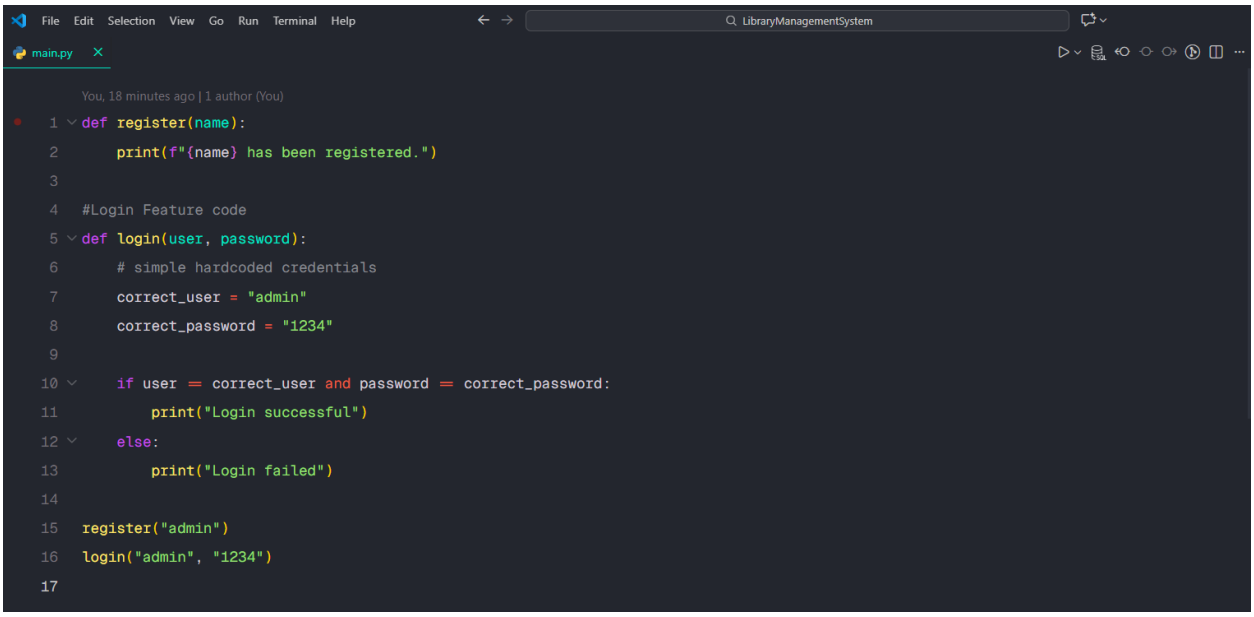
Switch to main branch

```
PS E:\Agile_DA\LibraryManagementSystem> git switch main
Switched to branch 'main'
```

Merge feature branch

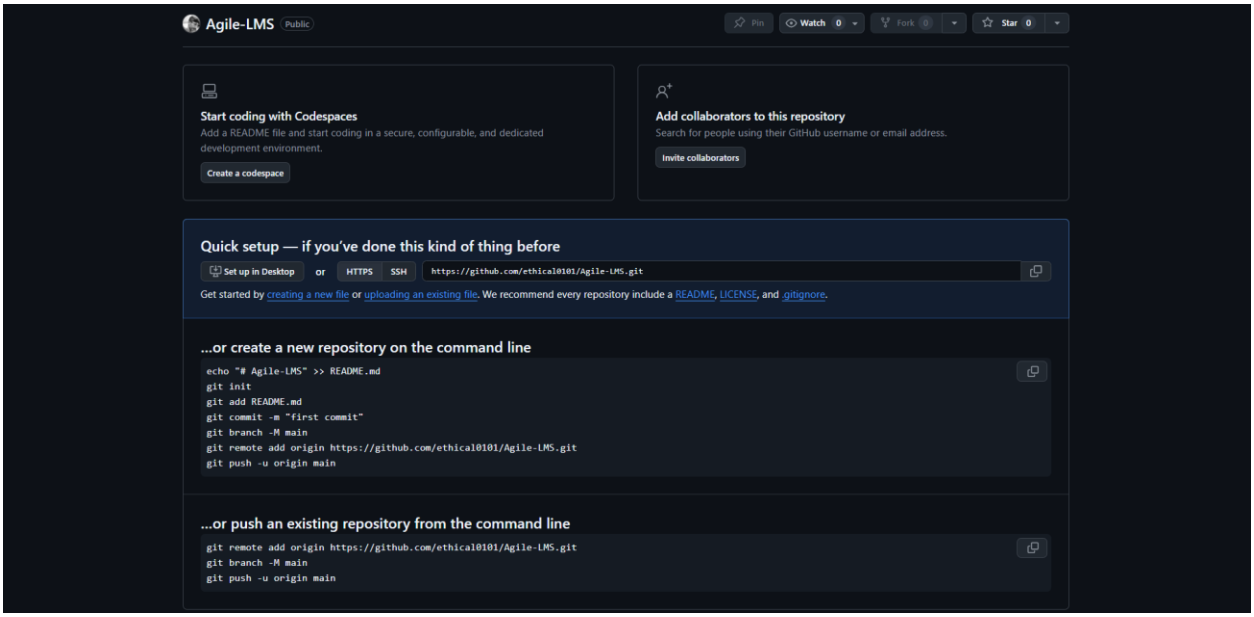
```
File  Edit  Selection  View  Go  Run  ...        LibraryManagementSystem
main.py
You, 18 minutes ago | 1 author (You)
def register(name):
    print(f"{name} has been registered.")

#Login Feature code
def login(user, password):
    # simple hardcoded credentials
    correct_user = "admin"
    correct_password = "1234"

    if user == correct_user and password == correct_password:

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS        powershell

PS E:\Agile_DA\LibraryManagementSystem> git switch main
Switched to branch 'main'
PS E:\Agile_DA\LibraryManagementSystem> git merge login-feature
Updating 755d3f5..a783f4f
Fast-forward
 main.py | 14 ++++++++++++++
 1 file changed, 14 insertions(+)
PS E:\Agile_DA\LibraryManagementSystem>
main  SVG Viewer  ethical0101/AI-Trip-Planner#14 needs reviewers  0 0  Connect  Live Share  BLACKBOX Agent  Open Website
```

Verify merged content(Reviewed & Verified)




**Case Study 5:** Working with Remote Repository Scenario: The project must be shared using GitHub.

Create remote repository



## Add remote origin

```
PS E:\Agile_DA\LibraryManagementSystem> git remote add origin https://github.com/ethical0101/Agile-LMS.git
PS E:\Agile_DA\LibraryManagementSystem> git branch -M main
```

## Push code to GitHub

```
PS E:\Agile_DA\LibraryManagementSystem> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 919 bytes | 306.00 KiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/ethical0101/Agile-LMS.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS E:\Agile_DA\LibraryManagementSystem>
```

**Case Study 6:** Undoing Mistakes Scenario: A file is modified incorrectly.

View file changes

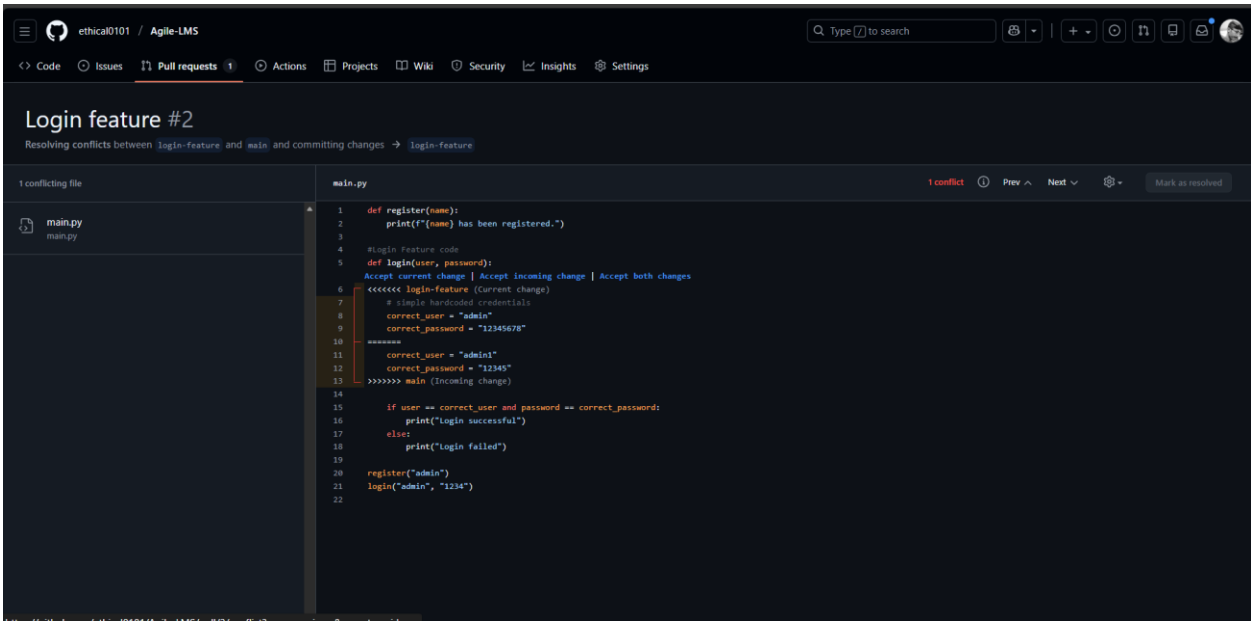
```
File  Edit  Selection  View  Go  Run  ...        LibraryManagementSystem
main.py

15  register("admin")
16  login("admin", "1234")
17
18  print("This is incorrect line")      You, now • Uncommitted changes

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS        powershell

PS E:\Agile_DA\LibraryManagementSystem> git status
        modified:   main.py

no changes added to commit (use "git add" and/or "git commit -a")
PS E:\Agile_DA\LibraryManagementSystem> git diff
diff --git a/main.py b/main.py
index dcd28b9..5707fc7 100644
--- a/main.py
+++ b/main.py
@@ -14,3 +14,5 @@ def login(user, password):

 register("admin")
 login("admin", "1234")
+
+print("This is incorrect line")
PS E:\Agile_DA\LibraryManagementSystem>
main*  SVG-Viewer  ethical0101/AI-Trip-Planner#14 needs reviewers  0 0  Connect  Live Share  BLACKBOX Agent  Open Website
```

Discard uncommitted changes

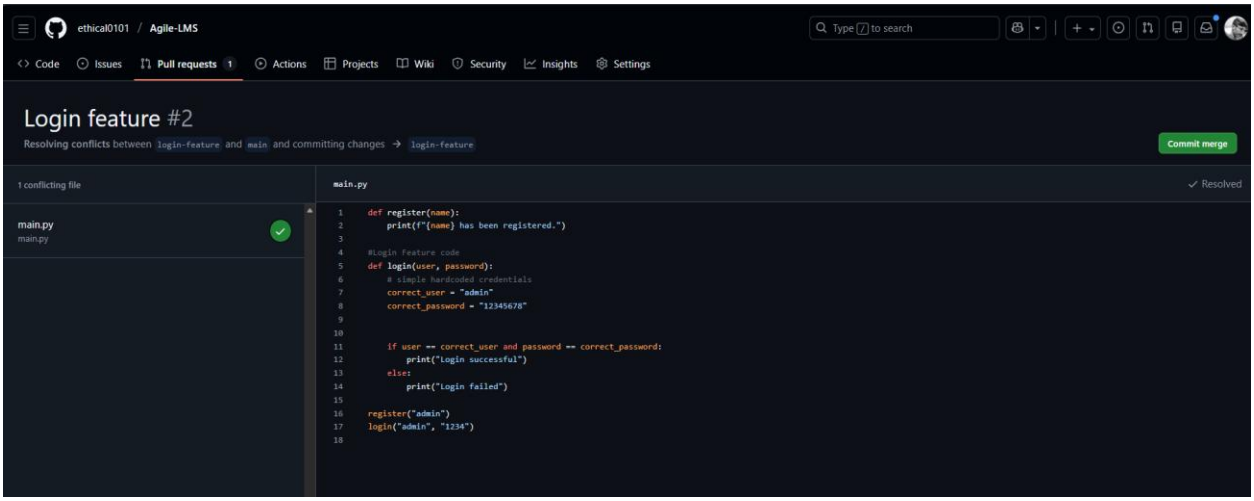
```
PS E:\Agile_DA\LibraryManagementSystem> git checkout -- main.py
PS E:\Agile_DA\LibraryManagementSystem>
```

**Case Study 7:** Handling Merge Conflicts Scenario: Two developers edited the same file.

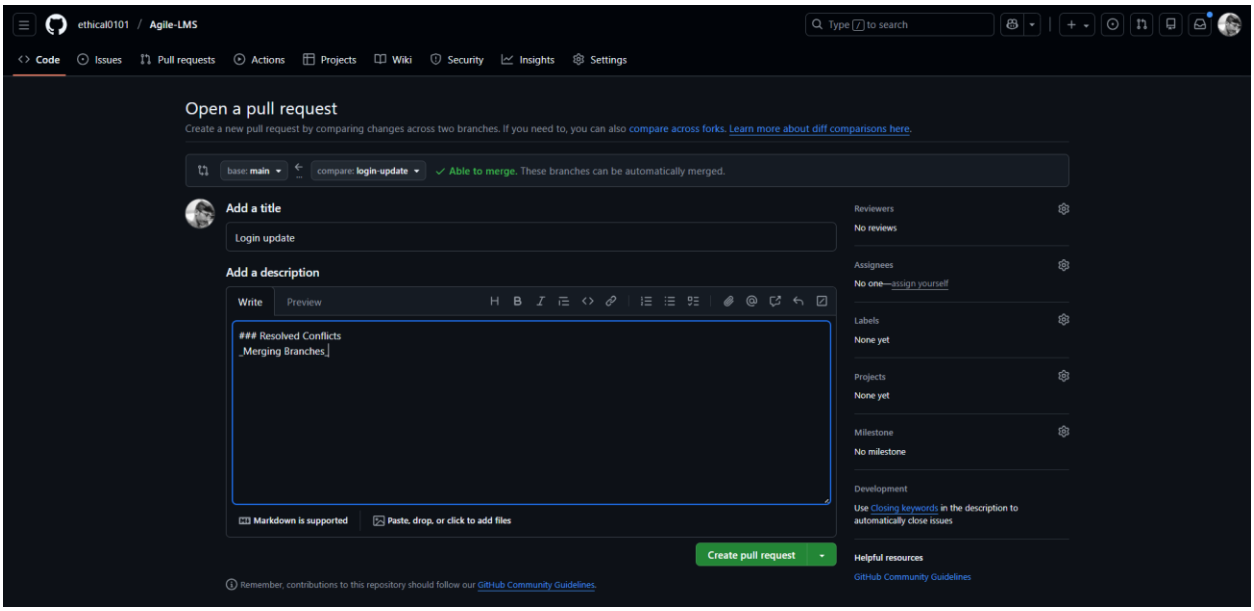
# Merge branches

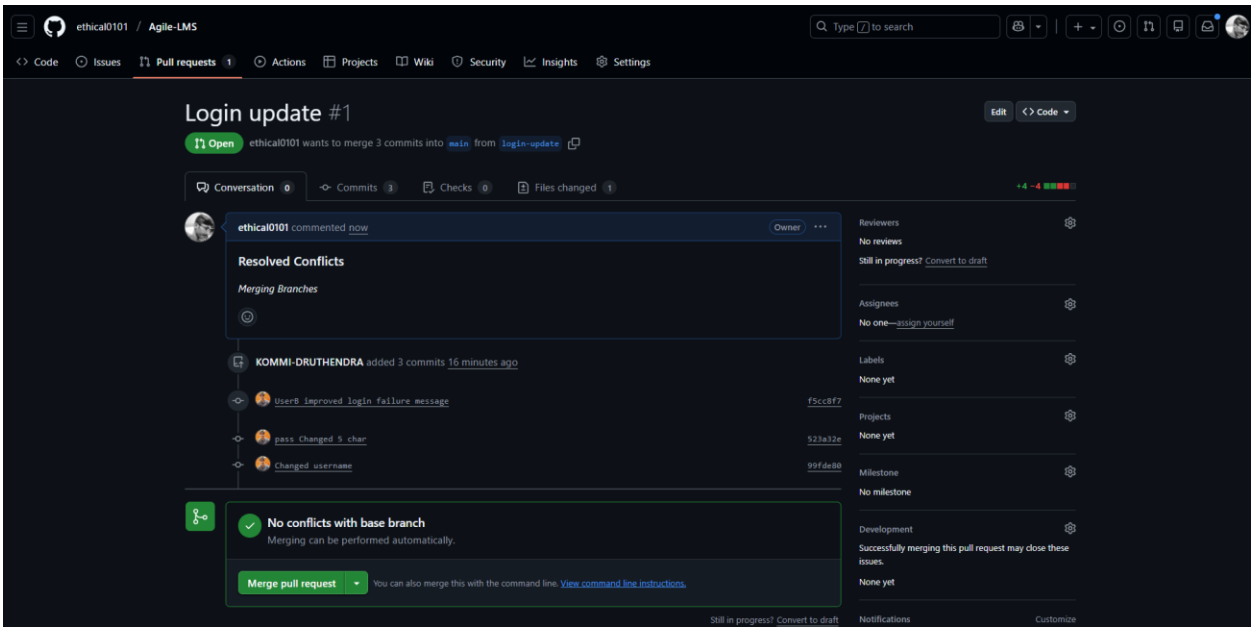



# Resolved Conflicts



## Merging Branches




**Case Study 8:** Ignoring Unwanted Files Scenario: Log and temporary files should not be tracked.

Create .gitignore file

```
PS E:\Agile_DA\LibraryManagementSystem> touch .gitignore
PS E:\Agile_DA\LibraryManagementSystem>
```

Add ignore patterns

```
*.log
*.tmp
__pycache__/
.env
```

Commit Changes

```
PS E:\Agile_DA\LibraryManagementSystem> git checkout main
Your branch is up to date with 'origin/main'.
PS E:\Agile_DA\LibraryManagementSystem> touch .gitignore
PS E:\Agile_DA\LibraryManagementSystem> git add .gitignore
PS E:\Agile_DA\LibraryManagementSystem> git commit -m "Ignored logs and temporary files"
[main d6eb37b] Ignored logs and temporary files
 1 file changed, 4 insertions(+)
 create mode 100644 .gitignore
PS E:\Agile_DA\LibraryManagementSystem>
```

**Case Study 9:** Collaborating with Team Scenario: A teammate pushed new changes.

Fetch latest changes

```
PS E:\Agile_DA\User_B\LMS_UserB> git fetch origin
PS E:\Agile_DA\User_B\LMS_UserB>
```

Pull Updates

```
PS E:\Agile_DA\User_B\LMS_UserB> git pull origin main
From https://github.com/ethical0101/Agile-LMS
 * branch            main       -> FETCH_HEAD
Updating eee59d3..7a83a88
Fast-forward
 main.py | 5 +++--
 1 file changed, 3 insertions(+), 2 deletions(-)
PS E:\Agile_DA\User_B\LMS_UserB>
```

Resolve conflicts if any

--No Conflicts

**Case Study 10:** Final Project Release Scenario: Project is ready for final submission.

Review commit history

```
PS E:\Agile_DA\LibraryManagementSystem> git log --oneline
d6eb37b (HEAD -> main) Ignored logs and temporary files
7a83a88 (origin/main, origin/HEAD) Merge pull request #2 from ethical0101/login-feature
0bcd2e6 (origin/login-feature) Merge branch 'main' into login-feature
5373ec1 Merge pull request #1 from ethical0101/login-update
99fde80 (origin/login-update) Changed username
a061347 (login-feature) Fixing Conflits
523a32e pass Changed 5 char
f20bae5 Enhanced Password to 8 digit
f5cc8f7 UserB improved login failure message
eee59d3 UserA improved register message
a783f4f Added Login Feature Code
755d3f5 Added student registration function
9cc5022 Initial commit
PS E:\Agile_DA\LibraryManagementSystem>
```

## Push final code

```
PS E:\Agile_DA\LibraryManagementSystem> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 328 bytes | 41.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ethical0101/Agile-LMS.git
   7a83a88..d6eb37b  main -> main
PS E:\Agile_DA\LibraryManagementSystem>
```

## Prepare release version

```
PS E:\Agile_DA\LibraryManagementSystem> git tag v1.0
PS E:\Agile_DA\LibraryManagementSystem> git push origin v1.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ethical0101/Agile-LMS.git
 * [new tag]         v1.0 -> v1.0
PS E:\Agile_DA\LibraryManagementSystem>
```

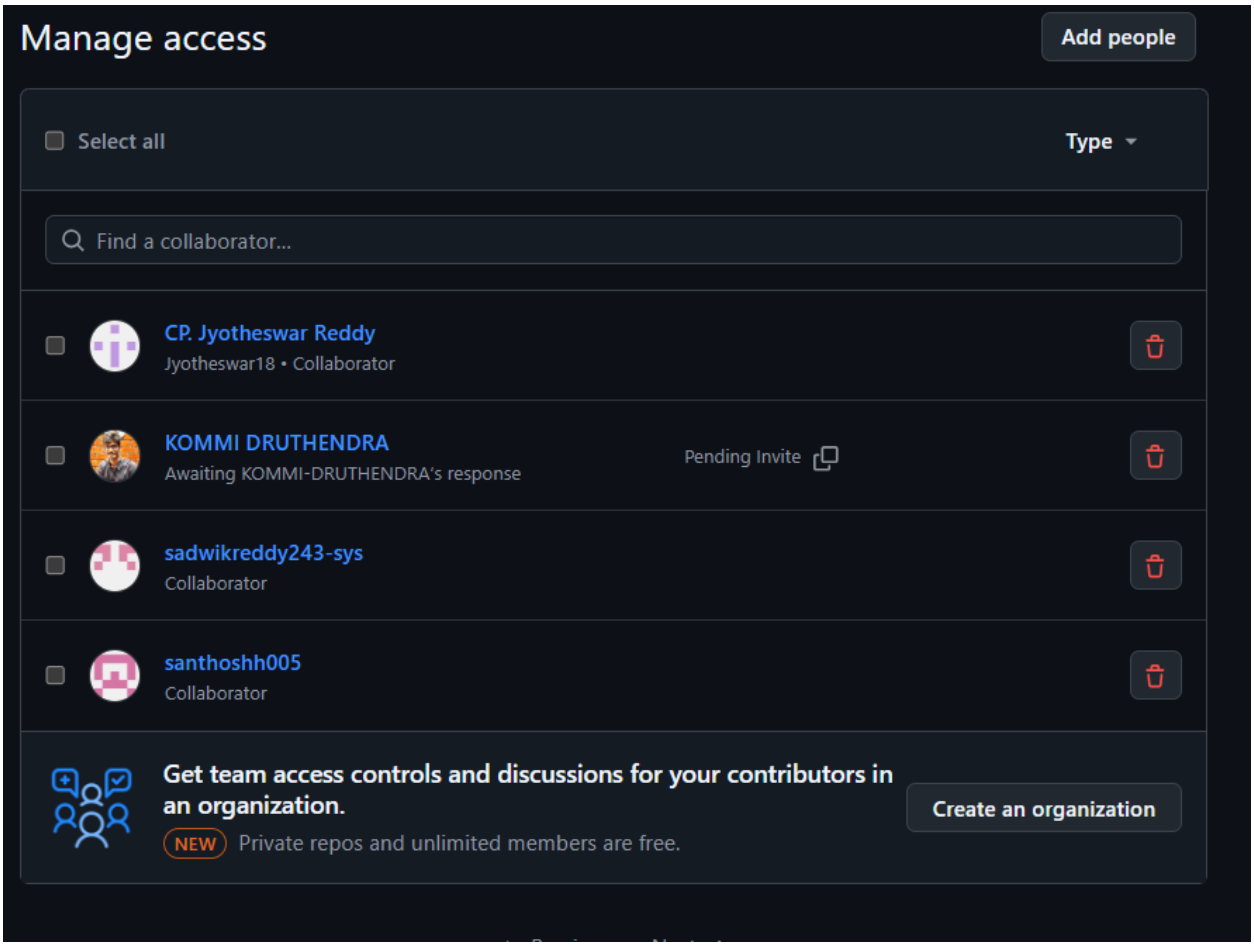
**Team Members :-**

Kommi Druthendra(23MIS0213)

G Sai Santhosh(23MIS0227)

CP Jyotheswar Reddy(23MIS0097)

G Sadwik Reddy(23MIS0541)




**Question 1.** A website requires the users to input username and password to register. Write a program to check the validity of password input by users. Following are the criteria for checking the password: 1. At least 1 letter between [a-z] 2. At least 1 number between [0-9] 1. At

least 1 letter between [A-Z] 3. At least 1 character from [$#@] 4. Minimum length of transaction password: 6 5. Maximum length of transaction password: 12 Your program should accept a sequence of comma separated passwords and will check them according to the above criteria. Passwords that match the criteria are to be printed, each separated by a comma.

Example If the following passwords are given as input to the program: ABd1234@1,a F1#,2w3E*,2We3345

Then, the output of the program should be: ABd1234@1

**Code:-**

File Edit Selection View Go Run Terminal Help

Question-1.py

```python
for pwd in passwords:
    if (6 <= len(pwd) <= 12 and
        re.search("[a-z]", pwd) and
        re.search("[A-Z]", pwd) and
        re.search("[0-9]", pwd) and
        re.search("[$#@]", pwd)):
        valid_passwords.append(pwd)

print(",".join(valid_passwords))
```

**Output:-**

```
PS E:\Agile_DA\LibraryManagementSystem> python .\Question-1.py
Enter comma separated passwords: ABd1234@1,a F1#,2w3E*,2We3345
ABd1234@1
PS E:\Agile_DA\LibraryManagementSystem>
```

**Question 2.** You are required to write a program to sort the (name, age, height) tuples by ascending order where name is string, age and height are numbers. The tuples are input by console. The sort criteria is: 1: Sort based on name; 2: Then sort based on age; 3: Then sort by score. The priority is that name > age > score. If the following tuples are given as input to the program: Tom,19,80 John,20,90 Jony,17,91 Jony,17,93 Json,21,85 Then, the output of the program should be:

[('John', '20', '90'),

('Jony', '17', '91'),

('Jony', '17', '93'),

('Json', '21', '85'),

('Tom', '19', '80')]

## Code:-

```python
data = []

print("Enter tuples (press Enter on empty line to stop):")

while True:
    line = input()
    if line == "":
        break
    name, age, height = line.split(",")
    data.append((name, age, height))

result = sorted(data, key=lambda x: (x[0], int(x[1]), int(x[2])))

print(result)
```

## Output:-

```
PS E:\Agile_DA\LibraryManagementSystem> python .\Question-2.py
Enter tuples (press Enter on empty line to stop):
Tom,19,80
John,20,90
Jony,17,91
Jony,17,93
Json,21,85

[('John', '20', '90'), ('Jony', '17', '91'), ('Jony', '17', '93'), ('Json', '21', '85'), ('Tom', '19', '80')]
PS E:\Agile_DA\LibraryManagementSystem>
```

**Question 3:** An electricity board calculates charges based on usage. Write a program that:

- Takes number of units consumed
- Calculates bill:
- First 100 units → ₹2/unit

- Next 100 units → ₹3/unit
- Remaining → ₹5/unit
- Displays total bill

```python
units = int(input("Enter units consumed: "))
bill = 0

if units ≤ 100:
    bill = units * 2
elif units ≤ 200:
    bill = (100 * 2) + ((units - 100) * 3)
else:
    bill = (100 * 2) + (100 * 3) + ((units - 200) * 5)

print("Total Electricity Bill: ₹", bill)
```

**Output:-**

```
PS E:\Agile_DA\LibraryManagementSystem> python .\Question-3.py
Enter units consumed: 2500
Total Electricity Bill: ₹ 12000
```

```
PS E:\Agile_DA\LibraryManagementSystem> python .\Question-3.py
Enter units consumed: 14089
Total Electricity Bill: ₹ 69945
```

**Question 4:** An ATM machine allows withdrawals only if sufficient balance is available.

Write a program that:

- Stores a user's balance
- Takes withdrawal amount as input

Checks:

- Amount must be a multiple of 100
- Balance must be sufficient
- Deducts amount and displays updated balance

**Code:-**

```python
balance = 10000
amount = int(input("Enter withdrawal amount: "))

if amount % 100 != 0:
    print("Amount must be a multiple of 100")
elif amount > balance:
    print("Insufficient balance")
else:
    balance -= amount
    print("Withdrawal successful")
    print("Updated Balance:", balance)
```

**Output:-**

```
PS E:\Agile_DA\LibraryManagementSystem> python .\Question-4.py
Enter withdrawal amount: 400
Withdrawal successful
Updated Balance: 9600
PS E:\Agile_DA\LibraryManagementSystem> python .\Question-4.py
Enter withdrawal amount: 11000
Insufficient balance
PS E:\Agile_DA\LibraryManagementSystem>
```

**Question 5:** A company calculates monthly salary based on basic pay. Write a program that:

- Accepts basic salary
- Calculates:
    - o HRA = 20% of basic
    - o DA = 10% of basic
    - o Tax = 5% of total salary
- Displays net salary

**Code:-**

```python
basic = float(input("Enter basic salary: "))

hra = 0.20 * basic
da = 0.10 * basic
gross_salary = basic + hra + da
tax = 0.05 * gross_salary
net_salary = gross_salary - tax

print("Net Salary:", net_salary)
```

**Output:-**

```
PS E:\Agile_DA\LibraryManagementSystem> python .\Question-5.py
Enter basic salary: 90000
Net Salary: 111150.0
PS E:\Agile_DA\LibraryManagementSystem> python .\Question-5.py
Enter basic salary: 2500000
Net Salary: 3087500.0
PS E:\Agile_DA\LibraryManagementSystem>
```